

Multimodal UX Discovery & Design Copilot

Product Requirements Document

Document Status: Draft
Version: 1.0
Date: September 28, 2026
Product Type: AI-powered UX reasoning and collaboration agent
Primary Surface: Figma/FigJam Plugin + Web Application

The product converts messy, multimodal product context into structured UX decisions, editable Figma/FigJam artifacts, and a conversational design collaboration experience.

Product Principle

“Do not jump from messy input directly to screens. First understand the context, expose assumptions, establish the UX structure, then collaborate with the designer to refine it.”

Contents

1	Product Overview	4
1.1	Product Name	4
1.2	Product Vision	4
2	Problem Statement	4
3	Goals	5
3.1	Primary Goals	5
4	Non-Goals	5
5	Target Users	6
5.1	Primary User — Product Designer	6
5.2	Product Manager	6
5.3	UX Researcher	6
5.4	UX/UI Lead	6
6	Core User Journey	7
6.1	Phase 1 — Automated Synthesis	7
6.1.1	Baseline Output	7
7	Phase 2 — Socratic Co-Pilot / Sparring Partner	8
8	Multimodal Input Requirements	8
9	Pre-Flight Check	8
9.1	Purpose	8
9.2	Example of Vague Input	9
9.3	Expected Behavior	9
9.4	Requirements	9
10	UX Artifact Generation	9
10.1	User Personas	9
10.2	Jobs-to-be-Done	10
10.3	Happy-Path User Flow	10
11	Assumption Management	10
11.1	Confirmed	10
11.2	Inferred	10
11.3	Unknown	11
11.4	Assumption	11
12	Figma/FigJam Integration	11
12.1	Primary Interface	11
12.2	Canvas Operations	11
12.3	Non-Destructive Editing	12
13	Design System Memory	12
14	Memory / RAG Requirements	12

15 Non-Disruptive Strategy Module	13
15.1 Purpose	13
15.2 Analysis Pipeline	13
15.3 Inputs	13
15.4 Outputs	13
15.4.1 Progressive Disclosure	14
15.4.2 Opt-In Toggle	14
15.4.3 Contextual Onboarding	14
15.4.4 Safe Fallback	14
15.5 Strategy Output Example	14
16 Conversational UX	14
16.1 Mode A — Pipeline	15
16.2 Mode B — Sparring	15
17 Agent Behavior	15
18 Source Traceability	16
19 Error Handling	16
19.1 Missing Input	16
19.2 Conflicting Requirements	16
19.3 Low-Confidence Visual Analysis	16
20 Functional Requirements	17
21 Non-Functional Requirements	17
21.1 Performance	17
21.2 Reliability	17
21.3 Security	17
21.4 Explainability	18
22 Proposed Technical Architecture	18
23 Proposed Agent Architecture	18
23.1 Orchestrator Agent	18
23.2 Context Extraction Agent	18
23.3 Pre-Flight Agent	19
23.4 UX Synthesis Agent	19
23.5 Sparring Agent	19
23.6 Flow Agent	19
23.7 Design-System Agent	19
23.8 Non-Disruptive Strategy Agent	19
23.9 Figma Execution Agent	19
24 Data Model	20
24.1 Project	20
24.2 Input	20
24.3 Evidence	20
24.4 Persona	20
24.5 JTBD	20
24.6 Flow	21
24.7 Assumption	21
24.8 Strategy	21

25 MVP Scope	21
25.1 MVP Input	21
25.2 MVP Pipeline	21
25.3 MVP Sparring	22
25.4 MVP Figma/FigJam	22
25.5 MVP Memory	22
25.6 MVP Non-Disruptive Strategy	22
26 Post-MVP Features	22
27 Success Metrics	23
27.1 Activation	23
27.2 UX Quality	23
27.3 Collaboration	23
27.4 Efficiency	23
27.5 Safety and Quality	23
28 Example End-to-End Workflow	23
29 Example User Interaction	24
29.1 Initial Request	24
29.2 Pre-Flight	24
29.3 Additional Context	25
29.4 Sparring Interaction	25
30 Acceptance Criteria	25
31 Product Principle	26

1 Product Overview

1.1 Product Name

Multimodal UX Discovery & Design Copilot

1.2 Product Vision

Build an AI design partner that can take a chaotic collection of product inputs—rough ideas, text prompts, PRDs, screenshots, competitor references, interview transcripts, existing UI references, and business constraints—and transform them into a structured UX strategy and editable Figma/FigJam workspace.

The product should not behave like a simple screen generator.

It should operate as a two-stage design partner:

1. **Pipeline:** Automatically synthesize unstructured inputs into a structured UX baseline.
2. **Sparring Partner:** Allow the designer to challenge assumptions, refine flows, explore alternatives, and update the generated workspace conversationally.

The system must also understand existing product/UI context and provide a **non-disruptive launch strategy** for introducing new features without unnecessarily disrupting established user behavior.

2 Problem Statement

UX teams receive information in fragmented formats:

- Rough product ideas
- Product requirements documents
- User interviews
- Competitor screenshots
- Existing product screenshots
- Business constraints
- Design-system documentation
- Product metrics
- Existing user flows

Current AI design tools often jump directly from a prompt to UI screens. This creates several problems:

- Important context is lost.
- AI makes assumptions when requirements are vague.
- Generated screens may conflict with the existing design system.
- Existing user behavior may be disrupted.
- Designers have limited ability to interrogate AI assumptions.
- AI may generate visually attractive screens without producing a defensible UX strategy.
- Introducing a new feature into an existing product requires more than generating a new screen.

The product therefore needs to act as a **context-aware UX reasoning and collaboration layer**, rather than only a UI generation tool.

3 Goals

3.1 Primary Goals

The product must:

1. Accept multimodal product and design inputs.
2. Automatically organize unstructured inputs into structured UX artifacts.
3. Generate a baseline UX workspace in Figma/FigJam.
4. Generate user personas.
5. Generate core Jobs-to-be-Done.
6. Generate a happy-path user flow.
7. Detect when the user's idea is too vague to safely execute.
8. Ask targeted clarification questions before generating a solution when required.
9. Provide an interactive AI sparring mode after the baseline is generated.
10. Allow designers to challenge assumptions and request alternatives.
11. Update the Figma/FigJam workspace based on conversational instructions.
12. Respect organizational design-system and product constraints.
13. Analyze existing UI references when provided.
14. Produce a non-disruptive feature rollout/deployment strategy.

4 Non-Goals

The MVP should **not** attempt to:

- Fully replace professional UX designers.
- Automatically make irreversible production changes.
- Automatically publish designs to production.
- Guarantee that generated UX decisions are correct.
- Generate an entire production-ready application from one prompt.
- Replace usability testing.
- Replace product analytics.
- Infer unsupported user needs as facts.
- Invent missing business requirements without clearly identifying assumptions.

5 Target Users

5.1 Primary User — Product Designer

Needs to:

- Quickly convert research into UX artifacts.
- Explore multiple user flows.
- Validate assumptions.
- Work directly inside Figma/FigJam.
- Preserve the existing design system.
- Iterate conversationally.

5.2 Product Manager

Needs to:

- Convert PRDs into actionable UX structures.
- Understand user goals.
- Explore feature implications.
- Validate happy paths.
- Understand rollout risks.

5.3 UX Researcher

Needs to:

- Convert interviews and research into structured insights.
- Connect user evidence to personas and JTBD.
- Preserve source context.
- Identify gaps in available evidence.

5.4 UX/UI Lead

Needs to:

- Review AI-generated UX decisions.
- Challenge assumptions.
- Maintain consistency with existing product patterns.
- Control how new functionality is introduced.

6 Core User Journey

6.1 Phase 1 — Automated Synthesis

The user provides a mixture of:

- Text prompt
- PRD
- Competitor screenshots
- Existing UI screenshots
- User interview transcripts
- Product requirements
- Business constraints
- Design-system documentation

The system then:

1. Ingests all available inputs.
2. Extracts text and visual information.
3. Classifies information by type.
4. Identifies explicit requirements.
5. Identifies constraints.
6. Identifies user needs.
7. Identifies existing UI patterns.
8. Detects contradictions.
9. Separates facts from assumptions.
10. Generates a structured UX baseline.

6.1.1 Baseline Output

The generated baseline workspace contains:

- 2–3 user personas
- Core Jobs-to-be-Done
- Happy-path user flow
- Key assumptions
- Constraints
- Open questions
- Initial UX structure

7 Phase 2 — Socratic Co-Pilot / Sparring Partner

Once the baseline has been generated, the conversational interface becomes active.

The designer can ask:

“This flow is too long. Can we simplify it to three steps?”

“Challenge my assumption that users want this feature on the dashboard.”

“Show me an alternative flow.”

The AI should:

1. Understand the current workspace.
2. Identify the affected UX artifact.
3. Explain the reasoning behind its response.
4. Suggest alternatives where appropriate.
5. Ask clarifying questions if necessary.
6. Apply approved changes to the Figma/FigJam workspace.
7. Preserve unaffected sections.

8 Multimodal Input Requirements

Input	Required	Purpose
Text prompt	Yes	Initial product/UX request
PRD	Yes	Requirements and constraints
Screenshots	Yes	Existing UI and competitor analysis
Interview transcript	Recommended	User evidence
Design-system documentation	Recommended	UI consistency
Existing Figma/FigJam references	Recommended	Product context
Business constraints	Recommended	Product limitations

Table 1: Supported Multimodal Inputs

The system must preserve the relationship between extracted information and its source.

Example:

Requirement

-> Source: PRD, Section 4

-> Confidence: High

9 Pre-Flight Check

9.1 Purpose

Prevent the LLM from inventing requirements when insufficient information is available.

9.2 Example of Vague Input

“Make a dashboard for tracking habits.”

This input does not provide enough information to safely generate a complete UX solution. The system should pause execution and enter **Sparring Partner mode**.

9.3 Expected Behavior

The agent should ask targeted questions:

- What constraints must the dashboard follow?
- What is the primary metric the dashboard needs to improve?
- Who is the target user?
- What decisions should the dashboard help the user make?
- Are there existing UI patterns that must be preserved?

Once sufficient information is collected, the pipeline resumes.

9.4 Requirements

The system must:

- Detect insufficient context.
- Explain what information is missing.
- Ask the minimum number of high-value questions.
- Avoid unnecessary questioning.
- Resume the pipeline once the minimum context threshold is met.

10 UX Artifact Generation

10.1 User Personas

Generate **2–3 personas** when sufficient evidence exists.

Each persona should include:

- Persona name/identifier
- Role
- Primary goal
- Relevant needs
- Pain points
- Context of use
- Supporting evidence
- Confidence level

The system must not invent unsupported demographic details.

10.2 Jobs-to-be-Done

The system should identify the core jobs users are trying to accomplish.

Suggested structure:

```
When [situation],  
I want to [motivation/action],  
so I can [expected outcome].
```

Each JTBD should contain:

- Job statement
- Supporting evidence
- Priority/context
- Confidence

10.3 Happy-Path User Flow

Generate the shortest valid path through the primary experience.

```
Entry  
|  
v  
Discover Feature  
|  
v  
Configure  
|  
v  
Confirm  
|  
v  
Complete
```

Each step should contain:

- User action
- System response
- Relevant state
- Required information
- Potential failure state

11 Assumption Management

The AI must distinguish between:

11.1 Confirmed

Directly supported by provided evidence.

11.2 Inferred

A reasonable interpretation based on available evidence.

11.3 Unknown

Information that is not available.

11.4 Assumption

A design hypothesis that requires validation.

Example:

ASSUMPTION

Users will prefer accessing the new feature from the dashboard.

Evidence

No direct evidence provided.

Validation

Test dashboard discoverability with existing users.

The system must never present an assumption as an established fact.

12 Figma/FigJam Integration

12.1 Primary Interface

The product should provide a Figma/FigJam Plugin.

The plugin should allow users to:

- Import context.
- Start the UX pipeline.
- View AI-generated artifacts.
- Chat with the AI.
- Apply changes to the workspace.
- Generate sticky notes.
- Generate connectors.
- Create structured flows.
- Update existing artifacts.

12.2 Canvas Operations

The system should be able to programmatically create and update:

- Frames
- Sticky notes
- Connectors
- Text blocks
- Flow diagrams

- Persona cards
- JTBD cards
- Assumption cards
- UX strategy sections

12.3 Non-Destructive Editing

AI updates should not overwrite unrelated designer work.

Before modifying an existing object:

1. Identify the target object.
2. Confirm the intended modification.
3. Modify only required properties.
4. Preserve designer-created content where possible.

13 Design System Memory

The system should maintain a persistent knowledge layer containing:

- Design-system guidelines
- Component rules
- Typography
- Colors
- Spacing
- Interaction patterns
- Accessibility requirements
- Product constraints
- Existing UX patterns
- Previously validated decisions

This knowledge should be retrieved when generating or modifying UX artifacts.

14 Memory / RAG Requirements

A vector database or equivalent retrieval system should store relevant product knowledge.

Potential technologies include:

- Pinecone
- Weaviate
- PostgreSQL + pgvector
- Other vector-capable databases

The system should support retrieval by:

- Product area
- Component
- Feature
- User type
- Requirement
- Design-system rule
- Existing UX pattern

The AI should retrieve relevant context before generating or modifying UX recommendations.

15 Non-Disruptive Strategy Module

15.1 Purpose

A major differentiator is the ability to determine how a new feature can be introduced without unnecessarily disrupting the existing product experience.

15.2 Analysis Pipeline

```
Existing Product Flow
    |
    v
New Feature Flow
    |
    v
Interaction Differences
    |
    v
Risk Areas
    |
    v
Rollout Strategy
```

15.3 Inputs

The module uses:

- Existing UI references
- Existing user flows
- New feature flow
- Design system
- Product constraints
- Known user behavior

15.4 Outputs

The module should produce a deployment strategy including:

15.4.1 Progressive Disclosure

Introduce advanced functionality gradually.

```
Existing UI
|
v
Optional Feature
|
v
Advanced Controls
```

15.4.2 Opt-In Toggle

Hide the new experience behind an explicit activation mechanism when appropriate.

15.4.3 Contextual Onboarding

Introduce the feature at the point where the user needs it.

15.4.4 Safe Fallback

Define behavior when:

- The new feature fails.
- Required data is missing.
- The user does not have access.
- The user chooses not to use the feature.

15.5 Strategy Output Example

Existing Flow Risk:
Medium

Primary Risk:
Introducing the feature at the first step adds complexity.

Suggested Strategy:
Keep the existing entry point unchanged and expose the new feature through progressive disclosure.

Fallback:
Return users to the existing workflow if the new feature cannot complete.

The system must clearly distinguish evidence-based findings from design hypotheses.

16 Conversational UX

The AI assistant should support two distinct interaction modes.

16.1 Mode A — Pipeline

Purpose:

Convert messy inputs into a structured baseline.

Characteristics:

- Automated
- Structured
- Batch-oriented
- Artifact-focused

16.2 Mode B — Sparring

Purpose:

Help the designer reason about and modify the baseline.

Characteristics:

- Conversational
- Socratic
- Iterative
- Critical
- Context-aware

17 Agent Behavior

The AI should:

- Challenge unsupported assumptions.
- Ask clarifying questions when necessary.
- Suggest alternatives.
- Identify trade-offs.
- Preserve user intent.
- Avoid hallucinating requirements.
- Reference source evidence.
- Explain uncertainty.
- Maintain consistency with existing design patterns.

The AI should not:

- Pretend assumptions are facts.
- Automatically overwrite designer work.
- Generate unnecessary complexity.
- Ignore existing product conventions.
- Make irreversible changes without confirmation.

18 Source Traceability

Every important generated insight should be traceable to one or more sources.

Example:

Insight:
Users need quick access to recent activity.

Source:
User Interview #12, timestamp 08:42

Confidence:
High

For inferred decisions:

Inference:
The feature may be easier to discover from the dashboard.

Basis:
Existing dashboard navigation + competitor reference.

Status:
Design hypothesis

19 Error Handling

19.1 Missing Input

Missing context detected.

Before I generate the flow, I need:

1. Target user
2. Primary goal
3. Existing product constraint

19.2 Conflicting Requirements

Requirement conflict detected.

Requirement A:
Feature must be accessible from the dashboard.

Requirement B:
Dashboard must remain unchanged.

Please clarify which constraint has priority.

19.3 Low-Confidence Visual Analysis

UI element could not be reliably identified.

Status:
Not Visible / Uncertain

Please provide a clearer reference if this detail is important.

20 Functional Requirements

ID	Requirement
FR-001	The system shall accept multiple input types in one session.
FR-002	The system shall analyze text and visual inputs together.
FR-003	The system shall classify information into requirements, constraints, user needs, existing patterns, evidence, assumptions, and unknowns.
FR-004	The system shall evaluate whether enough context exists before generating UX artifacts.
FR-005	The system shall ask targeted questions when context is insufficient.
FR-006	The system shall generate 2–3 evidence-backed personas when sufficient information exists.
FR-007	The system shall generate core Jobs-to-be-Done.
FR-008	The system shall generate a primary happy-path flow.
FR-009	The system shall create editable UX artifacts in Figma/FigJam.
FR-010	The system shall accept natural-language instructions for modifying generated artifacts.
FR-011	The system shall generate alternative flows when requested.
FR-012	The system shall identify unsupported assumptions.
FR-013	The system shall retrieve relevant design-system context before generating or modifying artifacts.
FR-014	The system shall analyze new flows against existing product references.
FR-015	The system shall generate rollout strategies and fallback states where applicable.
FR-016	The system shall preserve source references for important generated insights.
FR-017	The system shall avoid overwriting unrelated designer-created content.

21 Non-Functional Requirements

21.1 Performance

- Initial context processing should provide visible progress.
- Interactive conversational responses should stream where possible.
- Figma updates should be incremental rather than requiring full workspace regeneration.

21.2 Reliability

- Failed canvas operations must not corrupt existing work.
- Generated artifacts should be recoverable.
- AI-generated changes should have operation logs.

21.3 Security

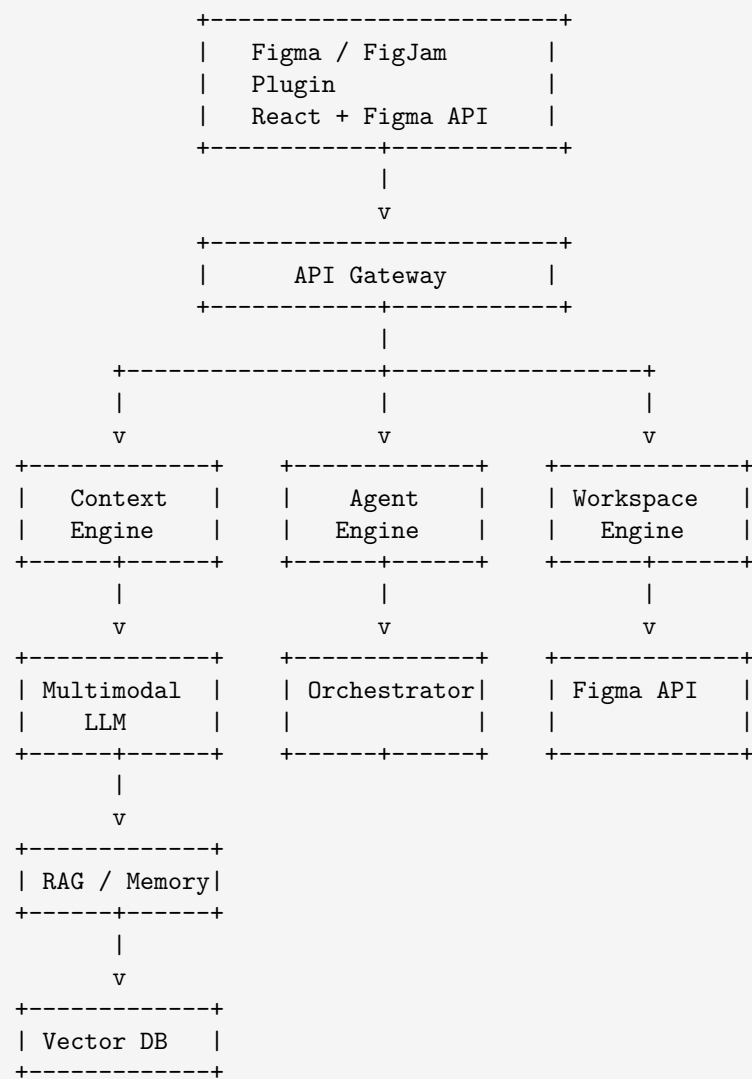
- Uploaded product documents may contain confidential information.
- Data must be isolated per workspace or organization.
- Access to external documents must be permission-controlled.
- Sensitive information should not be unnecessarily included in prompts.

21.4 Explainability

Important AI decisions should expose:

- Evidence
- Assumptions
- Confidence
- Rationale

22 Proposed Technical Architecture



23 Proposed Agent Architecture

23.1 Orchestrator Agent

Responsible for routing tasks and maintaining session state.

23.2 Context Extraction Agent

Extracts:

- Requirements
- Constraints
- Evidence
- User needs
- Existing UI patterns

23.3 Pre-Flight Agent

Determines whether the input is sufficiently specified.

23.4 UX Synthesis Agent

Generates:

- Personas
- JTBD
- Happy path
- Assumptions
- Open questions

23.5 Sparring Agent

Handles conversational reasoning and challenges assumptions.

23.6 Flow Agent

Creates and modifies user flows.

23.7 Design-System Agent

Retrieves and applies design-system constraints.

23.8 Non-Disruptive Strategy Agent

Analyzes:

- Existing flow
- New flow
- Interaction differences
- Adoption risks
- Rollout options
- Fallback strategies

23.9 Figma Execution Agent

Converts approved AI decisions into Figma/FigJam operations.

24 Data Model

24.1 Project

```
project_id  
name  
workspace_id  
created_at  
updated_at
```

24.2 Input

```
input_id  
project_id  
type  
source  
content  
metadata  
created_at
```

24.3 Evidence

```
evidence_id  
project_id  
source_input_id  
content  
category  
confidence
```

24.4 Persona

```
persona_id  
project_id  
name  
role  
goals  
needs  
pain_points  
evidence_ids  
confidence
```

24.5 JTBD

```
jtbd_id  
project_id  
situation  
motivation  
outcome  
evidence_ids  
confidence
```

24.6 Flow

```
flow_id
project_id
name
version
nodes
edges
assumptions
```

24.7 Assumption

```
assumption_id
project_id
statement
basis
confidence
validation_status
```

24.8 Strategy

```
strategy_id
project_id
existing_flow
new_flow
risks
recommendations
fallbacks
confidence
```

25 MVP Scope

The first version should remain focused.

25.1 MVP Input

- Text prompt
- PDF/PRD
- Images/screenshots

25.2 MVP Pipeline

- Context extraction
- Pre-flight check
- 2–3 personas
- JTBD
- Happy-path flow
- Assumptions and open questions

25.3 MVP Sparring

- Conversational questioning
- Flow simplification
- Alternative flow generation
- Assumption challenge

25.4 MVP Figma/FigJam

- Generate structured board
- Create sticky notes
- Create flow nodes
- Create connectors
- Update selected artifacts

25.5 MVP Memory

- Basic project-level RAG
- Design-system references
- Source traceability

25.6 MVP Non-Disruptive Strategy

- Compare existing and proposed flows
- Generate rollout strategy
- Generate fallback states

26 Post-MVP Features

Potential future capabilities:

- Full UI generation
- Interactive prototypes
- Usability testing integration
- Product analytics integration
- Experiment design
- A/B test hypothesis generation
- Automated accessibility analysis
- Design-token extraction
- Version comparison
- Team collaboration
- Design decision history

- Automatic research synthesis
- User-testing transcript analysis
- Production telemetry feedback loop

27 Success Metrics

27.1 Activation

- Percentage of sessions that successfully ingest multiple input types.
- Percentage of users completing the initial synthesis.

27.2 UX Quality

- Percentage of generated artifacts accepted or edited by designers.
- Percentage of generated flows requiring major restructuring.
- Number of AI-generated assumptions identified before implementation.

27.3 Collaboration

- Number of sparring interactions per project.
- Percentage of users who iterate on the baseline.

27.4 Efficiency

- Time from raw input to first usable UX board.
- Time saved compared with manual synthesis.

27.5 Safety and Quality

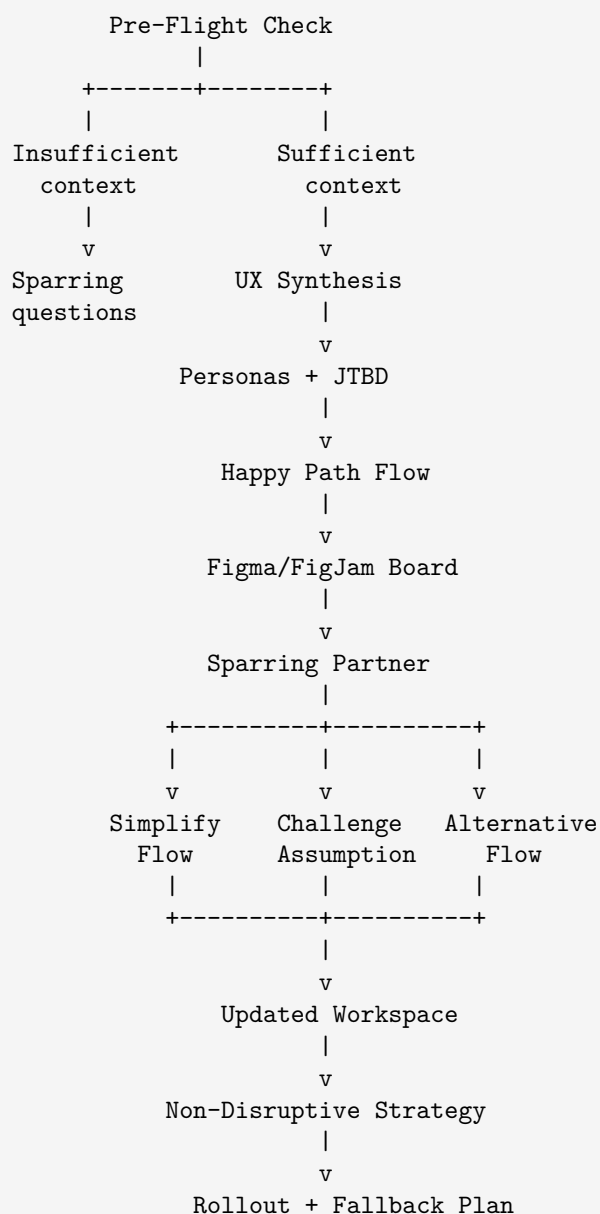
- Rate of unsupported assumptions presented as facts.
- Rate of destructive canvas modifications.
- Rate of design-system violations.
- Rate of incorrect source attribution.

28 Example End-to-End Workflow

```
User uploads:
|
+-- PRD.pdf
+-- 3 competitor screenshots
+-- 2 existing product screenshots
+-- user interview transcript
|
v

Context Extraction

|
v
```



29 Example User Interaction

29.1 Initial Request

“I want to add an AI assistant to the existing dashboard.”

29.2 Pre-Flight

The system detects missing information:

Before I build the flow, I need to clarify three things:

1. Who is the assistant primarily for?
2. What task should the assistant help users accomplish?
3. Should the existing dashboard navigation remain unchanged?

29.3 Additional Context

The user responds:

“It is for operations managers. It should answer questions about operational metrics. Keep the existing dashboard unchanged.”

The system then generates:

- Operations Manager persona
- JTBD
- AI assistant happy path
- Existing dashboard constraints
- Assumptions
- Open questions

29.4 Sparring Interaction

The user asks:

“Challenge the assumption that the assistant should be opened from the dashboard.”

The AI responds:

Assumption:

Dashboard entry point will maximize discoverability.

Counterpoint:

If users primarily ask questions after reviewing a metric, contextual entry points may reduce interaction cost.

Alternative:

Expose the assistant both globally and contextually, while preserving the existing dashboard navigation.

30 Acceptance Criteria

The MVP is considered functional when:

1. A user can upload a PRD, screenshots, and a prompt.
2. The system can extract structured context from those inputs.
3. The system detects insufficient information.
4. The system asks clarification questions instead of hallucinating requirements.
5. The system generates 2–3 personas.
6. The system generates JTBD.
7. The system generates a happy-path flow.
8. The generated artifacts can be placed into Figma/FigJam.
9. The user can challenge the generated flow conversationally.

10. The AI can produce an alternative flow.
11. The AI can update the workspace without destroying unrelated content.
12. The system can compare an existing flow with a proposed flow.
13. The system can produce a non-disruptive rollout strategy.
14. Important generated decisions can be traced back to source evidence.

31 Product Principle

First understand the context, expose assumptions, establish the UX structure, then collaborate with the designer to refine it. **Do not jump from messy input directly to screens. First understand the context, expose assumptions, establish the UX structure, then collaborate with the designer to refine it.**

The product is therefore not simply an AI UI generator.

It is a:

Multimodal UX reasoning and collaboration agent

that converts messy product context into structured design decisions and an editable workspace.